
Marketplace Engine

Pi connection defect register

Every connection defect found, ranked, with its current status

Version 1.0 · 13/09/2026

Repository: [bradroberts88/marketplace-engine-core](#)

Product codename: AutoPost

Contents

1.	Part 1 — The five that cause "identical cards, different outcome"
2.	Part 2 — Full register
3.	What is still open
4.	The one-line version

Every defect found in the 13/09/2026 connection review, in one list, with where it stands in the code as it exists today. Each entry was re-checked against the current files before its status was written — nothing here is marked fixed from memory.

The review found **26 numbered defects** across five areas plus **3 correctness notes**, 29 findings in total. The five that explain "two identical cards, one connects and one doesn't" are listed first because they account for almost every field failure.

Severity: **BLOCKER** = the card never connects. **FLAKY** = works on some hardware, images or boots. **RISK** = wrong or silent behaviour that hides the real cause.

Status: **Fixed** = closed in the current code. **Open** = still there. **By design** = behaves this way deliberately.

Source review, kept unchanged as the dated record: [docs/PI-CONNECT-FAILURE-ANALYSIS.md](#).

Part 1 — The five that cause "identical cards, different outcome"

A1 — A card that was never pre-registered can never register

BLOCKER · Fixed

What went wrong: the server only accepts a registration for a card that already exists. The card-prep script did not create that record — it printed a line of SQL at the end and trusted the operator to paste it. Miss that line on one card in ten and that card has Wi-Fi, has its tunnel, and is invisible to the fleet forever.

What you saw: **Registration failed. Retrying in 60s**, repeating for hours, on a card that looks perfectly healthy on a monitor.

Fix in place: prep now issues a single-use enrolment ticket to the server before it writes anything to the card, and aborts with **Card NOT written** if the server does not accept it. The card creates its own record on first boot. There is no manual SQL step left anywhere.

A2 — One Tailscale key reused across a batch

BLOCKER · Fixed

What went wrong: a single-use join key pasted into several cards. The first card joins; every other card loops on join failure and never reaches registration. Worse, the first card then blanks the key, so re-imaging even that card fails.

What you saw: **Tailscale join failed. Retrying in 60s** on all but one card of a batch.

Fix in place: prep mints a fresh tagged key per card automatically, and keeps a local ledger of key fingerprints — try to reuse one and the script refuses with the name of the card that already used it. The key is now blanked only after provisioning is recorded, so an interrupted card can still be re-imaged.

A3 — 2.4 GHz-only radios against 5 GHz dealer Wi-Fi

FLAKY · Fixed

What went wrong: a Pi Zero 2 W cannot see a 5 GHz network at all; a Pi 4 can. Same prepared file, same dealer, opposite outcome. Nothing reported "that network isn't in range" — the card just

waited and dropped to a rescue hotspot nobody looked for.

Fix in place: the card records its model and reports it with every check-in, and an out-of-range network is now reported specifically as `ssid_not_in_range_2g_radio` when the radio is the reason, so the fleet screen says what it is instead of "offline".

A4 — Boot-partition mount race

FLAKY · Fixed

What went wrong: the first-boot services checked for files on the boot partition without declaring that they needed it mounted. If systemd got there first, the service was skipped for the whole boot — not retried, skipped — and the journal showed only a cheerful "condition check resulted in ... being skipped". Timing-dependent, so it hit a minority of boots on a minority of cards.

Fix in place: every unit that reads the boot or data partition now declares the mount it depends on.

A5 — NetworkManager assumed but never required

BLOCKER · Fixed

What went wrong: every network action in the kit uses NetworkManager, and almost all of them silenced their own errors. On an older image without it, no dealer profile, no rescue hotspot, no setup page, and not one line in the log. The card boots, looks fine on a screen, and never connects.

Fix in place: first boot now checks NetworkManager is installed and running, starts it if it can, and fails loudly to the log and the screen if it cannot.

Part 2 — Full register

B — Boot and provisioning

ID	Severity	Where	Defect and fix	Status
B1	BLOCKER	<code>flash/provision-sd.sh</code>	The kernel line hardcoded the old <code>/boot</code> path while the first-boot script probes <code>/boot/firmware</code> first, so on a modern image first boot never ran and the card came up as stock Pi OS. Now the mount point is detected and written into the kernel line.	Fixed

ID	Severity	Where	Defect and fix	Status
B2	FLAKY	<code>flash/provision-sd.sh</code>	The token generator could be killed by a closed pipe mid-prep, and produced tokens shorter than intended. Replaced with a fixed-length hex generator that is length-checked before use.	Fixed
B3	RISK	<code>flash/provision-sd.sh</code>	A backend address typed with a trailing slash produced a double-slash URL that some gateways reject, so registration failed forever with no clue why. The slash is now stripped.	Fixed
B4	RISK	<code>boot-payload/qconnect-firststrun.sh</code>	The kernel-line cleanup matched one exact spelling; any change of ordering and the card re-ran first boot on every reboot. The cleanup now strips all three tokens by pattern.	Fixed
B5	RISK	<code>boot-payload/qconnect-firststrun.sh</code>	First boot ran with errors ignored and checked nothing, so a card missing its settings file ended up half-installed with no identity. Every install step is now checked and aborts with a readable message.	Fixed

ID	Severity	Where	Defect and fix	Status
B6	BLOCKER	golden-image build stage	The image installed the connector to one path while the service pointed at another, so cards from different image vintages behaved differently. The stage now repoints the service and refuses to finish if the paths don't agree.	Fixed

C — Wi-Fi and rescue

ID	Severity	Where	Defect and fix	Status
C1	FLAKY	<code>device/qconnect-portal.py</code>	The setup page scanned for networks while the radio was busy hosting the rescue hotspot — impossible on the Zero 2 W and Pi 3, so staff saw "found nothing" and gave up. A scan is now taken just before the hotspot comes up and cached for the page.	Fixed
C2	FLAKY	<code>device/qconnect-setup.sh</code>	The rescue hotspot was an open network, and iPhones drop an open network with no internet before the setup page ever opens. It is now WPA2-protected with a known password.	Fixed

ID	Severity	Where	Defect and fix	Status
C3	RISK	device/qconnect-setup.sh	A mistyped Wi-Fi password was never verified: the page closed, the card sat offline for 13 minutes, and nobody was told. New credentials are now verified and the page reports "that password was not accepted".	Fixed
C4	RISK	device/qconnect-setup.sh	A dealer guest portal blocking the card was written to the card's own disk only, so nobody upstream ever learned why. The block reason now travels with the check-in.	Fixed
C5	RISK	device/qconnect-setup.sh	A long card name produced a hotspot name over the 32-byte limit, which fails silently — no rescue network at all. The name is now capped.	Fixed
C6	RISK	first boot	The Wi-Fi country was set only through a tool that is absent on minimal images, leaving the radio possibly blocked. It is now written directly and the radio explicitly unblocked.	Fixed
C7	RISK	Wi-Fi profile creation	Hidden dealer networks were not supported at all and could never associate. Hidden is now a prep option and a tick box on the rescue page.	Fixed

D — Server and control channel (older AutoPost path)

ID	Severity	Where	Defect and fix	Status
D1	FLAKY	<code>connector/deploy/pi/autopost-connector.service</code>	Same mount race as A4 on the data partition: the unit could be skipped for the whole boot and the dealership showed offline until someone rebooted it. The mount dependency is now declared.	Fixed
D2	RISK	<code>connector/src/agent.js</code>	A dealership on a poor uplink that misses two check-ins is marked not-live and requests fail closed. Correct behaviour, but on screen it is indistinguishable from a broken Pi. Fail-closed is deliberate; the fleet screen is where this gets explained, not the agent.	By design
D3	RISK	<code>connector/src/claim.js</code>	A re-flashed card using a spent setup code looked exactly like a network failure in the log. The refusal reason and status are now carried on the error. A spent code still needs a new one minted — that part is intentional.	Fixed

ID	Severity	Where	Defect and fix	Status
D4	RISK	<code>connector/deploy/pi/autopost-claim.service</code>	The claim service runs as a limited user but reads its settings from the boot partition. On a card whose boot partition is mounted with restrictive permissions the unit fails with a confusing "failed to load environment files" instead of a claim error. Still present on the older AutoPost path; the new kit does not use this pattern.	Open

E — Telemetry that hid all of the above

ID	Severity	Where	Defect and fix	Status
E1	BLOCKER for diagnosis	<code>device/qconnect-heartbeat.sh</code>	The check-in threw away the server's answer and always reported success. A card with a bad token checked in forever into the void and showed as "never seen". The response is now read, logged, and a rejection is recorded on the card and surfaced.	Fixed
E2	RISK	<code>device/qconnect-heartbeat.sh</code>	Temperature and free disk defaulted to zero when the read failed, so a dead sensor looked like a healthy zero. Unreadable values now report as nothing at all.	Fixed

ID	Severity	Where	Defect and fix	Status
E3	RISK	heartbeat timer	The first check-in was attempted two minutes after boot, long before setup can finish, and was skipped — so the fleet view stayed empty long after the card was fine. Check-ins now start at 45 seconds and repeat every five minutes, and the card reports itself even before setup completes.	Fixed
E4	RISK	device/qconnect-setup.sh	Everything went to a log on the card, so "alive but stuck at step 4" and "dead" looked the same without physically going to the card. The card now reports which step it is stuck on with every check-in.	Fixed

F — Correctness notes (not connection failures)

ID	Severity	Where	Note and status	Status
F1	RISK	registration and check-in endpoints	These are open endpoints with the device token as the only credential, which is correct for a device that has no user — but token rotation is then the only way to revoke a card. Paired with the admin kill switch and audit log so a card can be cut off immediately.	Mitigated

ID	Severity	Where	Note and status	Status
F2	RISK	tunnel join	The join key was blanked before registration succeeded, so a card powered off in between and later re-imaged had no key left. Blanking now happens only after provisioning is recorded.	Fixed
F3	RISK	<code>device/qconnect-portal.py</code>	The setup page listened on every network interface, so during the brief overlap between the dealer network and the rescue hotspot it was reachable on the dealer LAN. It now binds to the hotspot address only, with a fallback to the old behaviour if that address isn't up yet — the fallback is the remaining exposure.	Mostly fixed

What is still open

1. **D4** — the older AutoPost claim service reads its settings from the boot partition as a limited user, which fails confusingly on a card whose boot partition is mounted with restrictive permissions. Only affects the older path; the new kit doesn't use it.
2. **F3** — the rescue setup page falls back to listening on every interface if the hotspot address isn't up yet. Narrow window, but it is a window.
3. **D2** — deliberate. A dealership that misses two check-ins fails closed. Worth a clearer message on the fleet screen rather than a code change.

Everything else found in the review is closed in the current code.

The one-line version

Two cards can be byte-for-byte identical and behave differently for exactly five reasons: one was never registered, they shared a join key, one has a radio that can't see the network, one lost a boot-timing race, or one is on an image the kit silently doesn't support. All five are now closed, and four of the five now fail loudly instead of quietly.